MANAGING DATABASE SCHEMAS WITH FLYWAY

Spring Data R2DBC supports initializing data sources through schema.sql and data.sql files, just like Spring Data JDBC. As you learned in chapter 5, that functionality is convenient for demos and experiments, but it's better to manage schemas explicitly for production use cases.

For Catalog Service we used Flyway to create and evolve its database schemas. We can do the same for Order Service. However, Flyway doesn't support R2DBC yet, so we need to provide a JDBC driver to communicate with the database. The Flyway migration tasks are only run at application startup and in a single thread, so using a non-reactive communication approach for this one case doesn't impact the overall application's scalability and efficiency.

In the build.gradle file for your Order Service project, add new dependencies to Flyway, the PostgreSQL JDBC driver, and Spring JDBC. Remember to refresh or reimport the Gradle dependencies after the new addition.

Listing 8.9 Adding a dependency for Flyway and JDBC in Order Service

```
dependencies {
  ...
  runtimeOnly 'org.flywaydb:flyway-core'              ❶
  runtimeOnly 'org.postgresql:postgresql'             ❷
  runtimeOnly 'org.springframework:spring-jdbc'       ❸
}
```

❶ Provides functionality to version-control your database through migrations

❷ Provides a JDBC driver that allows the application to connect to a PostgreSQL database

❸ Provides Spring integration with the JDBC API. It's part of the Spring Framework, not to be confused with Spring Data JDBC.